



NAVIS SMART INDUSTRY — Tổng quan hệ thống

Mục đích: Tài liệu này mô tả toàn bộ hệ thống Navis Smart Industry — dành cho AI đọc hiểu context, hoặc nhân viên mới vào công ty nắm bắt nhanh.

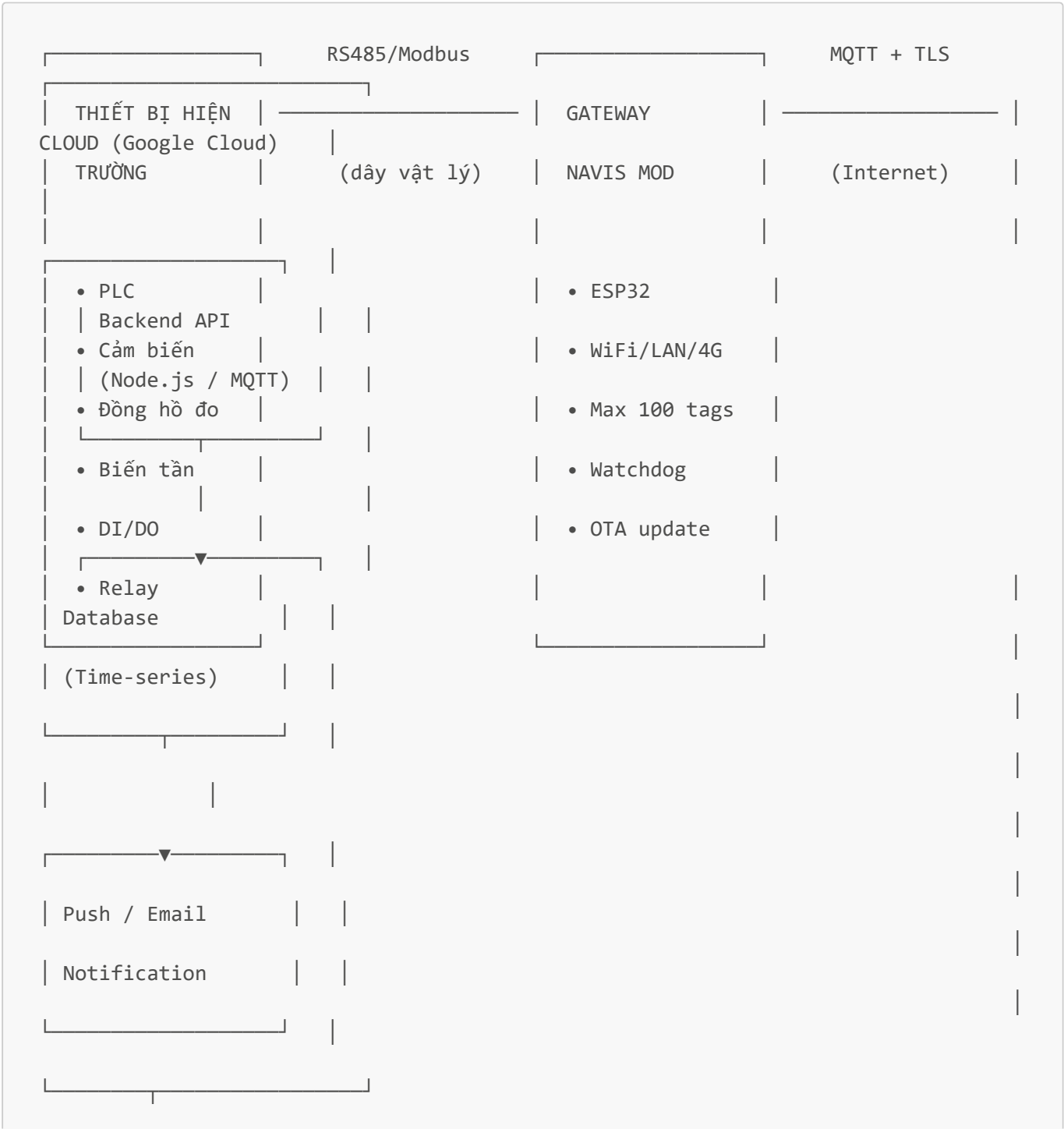
Cập nhật: Tháng 06/2026

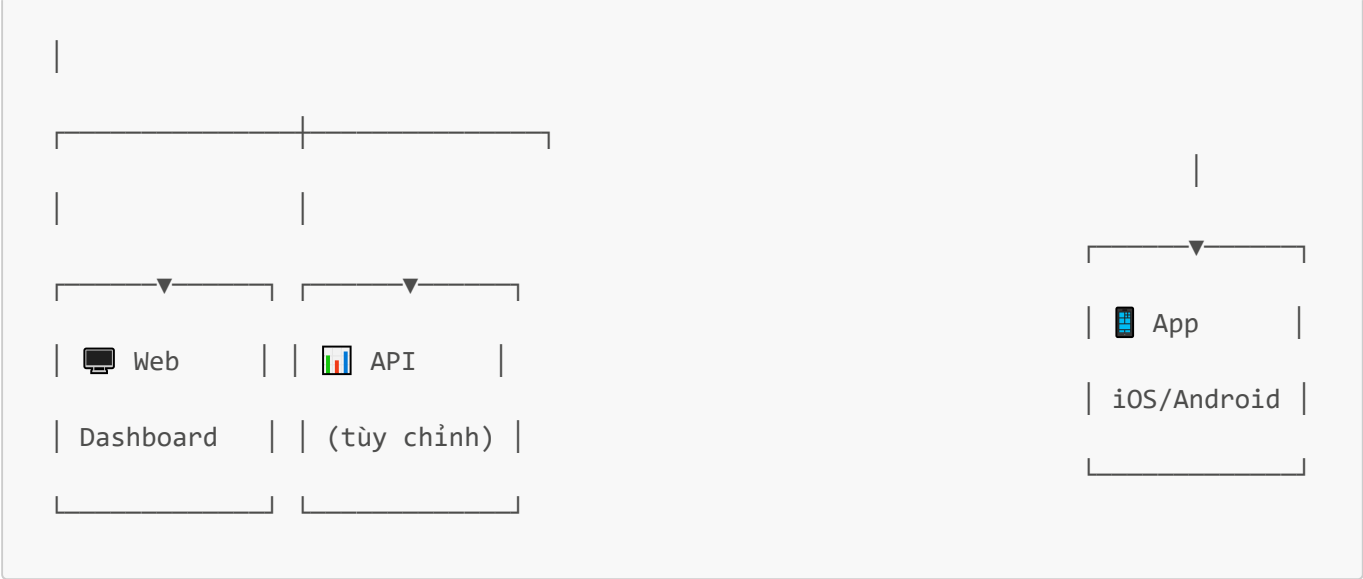
Website: navis.iotdaiviet.com

Công ty: Đại Việt IoT — Chuyên giải pháp IoT công nghiệp tại Việt Nam



Kiến trúc tổng thể





1. FIRMWARE — Gateway Navis MOD

1.1 Phần cứng

Thông số	Chi tiết
Vi xử lý	ESP32 (dual-core)
Kích thước	10 × 7.5 × 3 cm
Giao tiếp hiện trường	RS485 Modbus RTU (chuẩn công nghiệp)
Kết nối Internet	WiFi 2.4GHz / LAN Ethernet / 4G LTE (tùy phiên bản)
Nguồn cấp	12-24VDC (nguồn công nghiệp)
Số lượng tag tối đa	100 tags / gateway
Bảo vệ	Watchdog tự reset, chống treo
Cập nhật	OTA (Over-The-Air) — update firmware từ xa
Giao thức lên Cloud	MQTT + mã hóa TLS/SSL + JWT authentication
Đèn LED	WiFi (xanh dương): 2 lần/s = đang kết nối WiFi, 1 lần/s = LAN, 3 lần/s = 4G, sáng liên tục = đã kết nối
Bảo hành	12 tháng từ ngày nghiệm thu
Khoảng cách RS485	Tối đa 1200m (daisy-chain nhiều slave)

1.2 Chức năng Gateway

- **Đọc dữ liệu** từ thiết bị Modbus RTU (PLC, cảm biến, inverter, đồng hồ đo...)

- **Ghi dữ liệu** ngược lại thiết bị (điều khiển bật/tắt relay, set giá trị)
- **Đẩy dữ liệu** lên Cloud server qua MQTT realtime
- **Tự phục hồi** khi mất kết nối (watchdog, auto-reconnect)
- **Cấu hình qua App**: quét QR → cài đặt Modbus address, baud rate, data type...
- **Không cần lập trình**: người dùng chỉ cần điền thông số Modbus trên App
- **Kết nối WiFi**: Giữ nút Config >5s → kết nối qua App
- **Kết nối LAN**: Cắm RJ45 → nhấn nút Config 1 lần → auto DHCP
- **Kết nối 4G**: Lắp SIM → tự động kết nối (chỉ Vietnam: Viettel/Mobi/Vina)

1.3 Thiết bị tương thích

- PLC (Siemens, Mitsubishi, Delta, Omron...)
- Cảm biến: pH, COD, DO, nhiệt độ, áp suất, lưu lượng
- Đồng hồ đo điện (V, A, kW, kWh)
- Biến tần (inverter)
- Module DI/DO
- Camera (RTSP)
- Auto-sampler
- Bất kỳ thiết bị nào có RS485/Modbus
- 4-20mA / 0-10V: hỗ trợ gián tiếp qua converter analog-to-Modbus

1.4 Hạn chế hiện tại

- ⚠ **Không lưu trữ dữ liệu cục bộ** khi mất internet (đọc→gửi trực tiếp)
- ⚠ **Không có offline buffering** (backfill khi có lại mạng — planned for future)
- ⚠ **4G chỉ hoạt động tại Việt Nam**

1.5 Các loại dữ liệu Modbus hỗ trợ

Loại	Function Code	Mô tả	Ví dụ
SENSOR	FC03 (Read Holding) / FC04 (Read Input)	Đọc giá trị analog	Nhiệt độ, áp suất, lưu lượng, pH, DO
SETTING	FC06 (Write Single) / FC16 (Write Multiple)	Ghi giá trị xuống thiết bị	Set tốc độ biến tần, set ngưỡng
COIL	FC01 (Read Coil) / FC05 (Write Coil)	Đọc/ghi trạng thái ON/OFF	Bật/tắt bơm, van, relay
ALARM	FC01/FC02	Đọc trạng thái cảnh báo	Báo cháy, mức cao, lỗi thiết bị

1.6 Các data type hỗ trợ

- **INT16, UINT16** (16-bit)
- **INT32, UINT32** (32-bit, byte order: AB CD hoặc CD AB)
- **FLOAT32** (IEEE 754, byte order: AB CD hoặc CD AB)
- Hỗ trợ scale (nhân/chia hệ số) và offset

1.7 Tài liệu liên quan

- **Hướng dẫn cài đặt Gateway:** modbus-guide/index.html
- **Video cài đặt:** modbus-guide/NAVIS_MOD_Cai_Dat.mp4
- **Trang Gateway Setup:** docs/gateway-setup.html

2. APP MOBILE — Navis IoT

2.1 Thông tin chung

Thông số	Chi tiết
Tên App	Navis Smart Industry (tìm "Đại Việt IoT")
Nền tảng	iOS (App Store) + Android (Google Play)
Framework	Flutter (cross-platform)
Đăng nhập	Email+Password, Google, Apple ID (iOS), Phone+OTP
Ngôn ngữ	Hỗ trợ Tiếng Việt và English
Download	App Store / Google Play
QR Download	https://onelink.to/3nga3q

2.2 Tính năng App

Dashboard Realtime

- Hiển thị tất cả thông số cảm biến trên 1 màn hình
- Cập nhật liên tục (realtime qua MQTT)
- Hiển thị giá trị hiện tại, unit, trạng thái online/offline
- Card layout responsive, kéo sắp xếp thứ tự

Biểu đồ & Thống kê

- Line chart zoom được (pinch to zoom)
- Lọc theo khoảng thời gian: ngày / tuần / tháng / tùy chỉnh
- Tính tự động: Min, Max, Average
- Tính tổng tích lũy: kWh, m³ (cho điện năng, lưu lượng)
- Overlay nhiều thông số trên 1 biểu đồ

Bản đồ giám sát

- Hiển thị tất cả trạm trên Google Maps
- Marker màu theo trạng thái:
 -  Xanh = Bình thường
 -  Đỏ = Có cảnh báo
 -  Vàng = Mất kết nối
- Click marker → xem chi tiết trạm

Cảnh báo Push Notification

- Cài ngưỡng cao/thấp cho từng thông số
- Delay chống báo giả (ví dụ: vượt ngưỡng 30 giây mới báo)
- Push notification tức thì lên điện thoại
- Email cảnh báo (tùy chọn)
- Gọi điện thoại khi sự cố nghiêm trọng (tùy chọn)
- Zalo ZNS notification (tùy chọn)

Phân quyền

- 4 cấp quyền:
 - **Owner:** Toàn quyền, quản lý thanh toán
 - **Admin:** Quản lý thiết bị, người dùng
 - **Kỹ thuật:** Xem + điều khiển
 - **Viewer:** Chỉ xem, không điều khiển
- Phân quyền theo từng trạm riêng biệt
- 1 tài khoản có thể truy cập nhiều trạm

Các tính năng khác

- **Quét QR** để thêm thiết bị gateway
- **Điều khiển bật/tắt** relay/van từ xa (COIL)
- **Set giá trị** (SETTING) từ xa
- **Xem lịch sử cảnh báo** theo timeline
- **Chia sẻ trạm** cho đồng nghiệp bằng QR/link
- **Đa ngôn ngữ:** Tiếng Việt / English
- **Smart App Banner:** popup mời tải app khi vào web từ mobile

3. WEB DASHBOARD

3.1 Thông tin

Thông số	Chi tiết
URL chung	https://navis.iotdaiviet.com
Domain riêng	Có thể cấu hình domain riêng cho khách hàng (VD: scada.tenconty.com)
Responsive	Hoạt động trên PC, tablet, mobile
Trình duyệt	Chrome, Safari, Firefox, Edge

3.2 Tính năng Web

- **Dashboard tổng quan:** Giống app nhưng trên màn hình lớn hơn
- **Biểu đồ chi tiết:** Line chart lớn, so sánh nhiều thông số
- **Bản đồ trạm:** Google Maps fullscreen
- **P&ID (Sơ đồ công nghệ):** Hiển thị sơ đồ hệ thống SVG với giá trị realtime

- **Xuất báo cáo:**
 - Excel (.xlsx) với header logo công ty
 - PDF với biểu đồ nhúng
 - Gửi email tự động hàng ngày/tuần/tháng
- **Quản lý người dùng:** Thêm/xóa/phân quyền
- **Cấu hình thiết bị:** Thêm/sửa tag Modbus
- **Lịch sử cảnh báo:** Bảng log, filter theo ngày/trạm/mức độ

4. ☁ SERVER & CLOUD

4.1 Hạ tầng

Thông số	Chi tiết
Cloud Provider	Google Cloud Platform (GCP)
Region	Châu Á (Singapore / Hong Kong) — latency thấp cho VN
Đại lý	Đại lý chính thức Google Cloud tại VN — có xuất VAT đầy đủ
Mã hóa	TLS/SSL cho mọi kết nối
Backup	Tự động backup hàng ngày
Uptime	99.9% SLA

4.2 Kiến trúc Server

- **MQTT Broker:** Nhận dữ liệu realtime từ gateway
- **API Server:** RESTful API cho App/Web
- **Database:** Time-series database lưu trữ dữ liệu cảm biến
- **Push Service:** Firebase Cloud Messaging (FCM) cho push notification
- **Email Service:** Gửi cảnh báo và báo cáo tự động
- **File Storage:** Lưu trữ báo cáo PDF/Excel đã tạo

4.3 Bảo mật

- Mã hóa TLS/SSL end-to-end
- Xác thực token cho mọi API call
- Phân quyền 4 cấp (Owner/Admin/Kỹ thuật/Viewer)
- Không lưu mật khẩu dạng plain text (bcrypt hash)
- Audit log cho các thao tác quan trọng

5. 💰 MÔ HÌNH ĐỊNH GIÁ

5.1 Phí thiết bị (một lần)

Gói	Nội dung	Ghi chú
Gateway Navis MOD	Thiết bị gateway phần cứng	Mua 1 lần, sở hữu vĩnh viễn

Gói	Nội dung	Ghi chú
Phụ kiện	Anten WiFi/4G, nguồn 24V, dây RS485	Tùy cấu hình

5.2 Phí dịch vụ Cloud — 5,500,000 VNĐ/năm/trạm

Hạng mục	Mô tả
Server Cloud	Google Cloud (có VAT đầy đủ)
App + Web	✅ Bao gồm (iOS + Android + Web)
Lưu trữ dữ liệu	✅ Lưu trữ lịch sử dài hạn
Cảnh báo Push + Email	✅ Không giới hạn
Báo cáo tự động	✅ Excel/PDF + email tự động
Hỗ trợ kỹ thuật từ xa	✅ Bao gồm
Cập nhật phần mềm	✅ Bao gồm
SSL Certificate	✅ Miễn phí, tự động

Khách hàng tự chuẩn bị:

- ❌ Tên miền riêng (nếu muốn domain riêng)
- ❌ SIM 4G (nếu dùng 4G, chỉ Vietnam)

5.3 Gói tùy chọn

Gói	Mô tả
Domain riêng	Web dashboard trên domain công ty khách
P&ID tùy chỉnh	Thiết kế sơ đồ công nghệ theo yêu cầu
Zalo ZNS	Gửi cảnh báo qua Zalo (phí theo tin nhắn)
API tích hợp	API riêng cho khách tích hợp vào hệ thống sẵn có

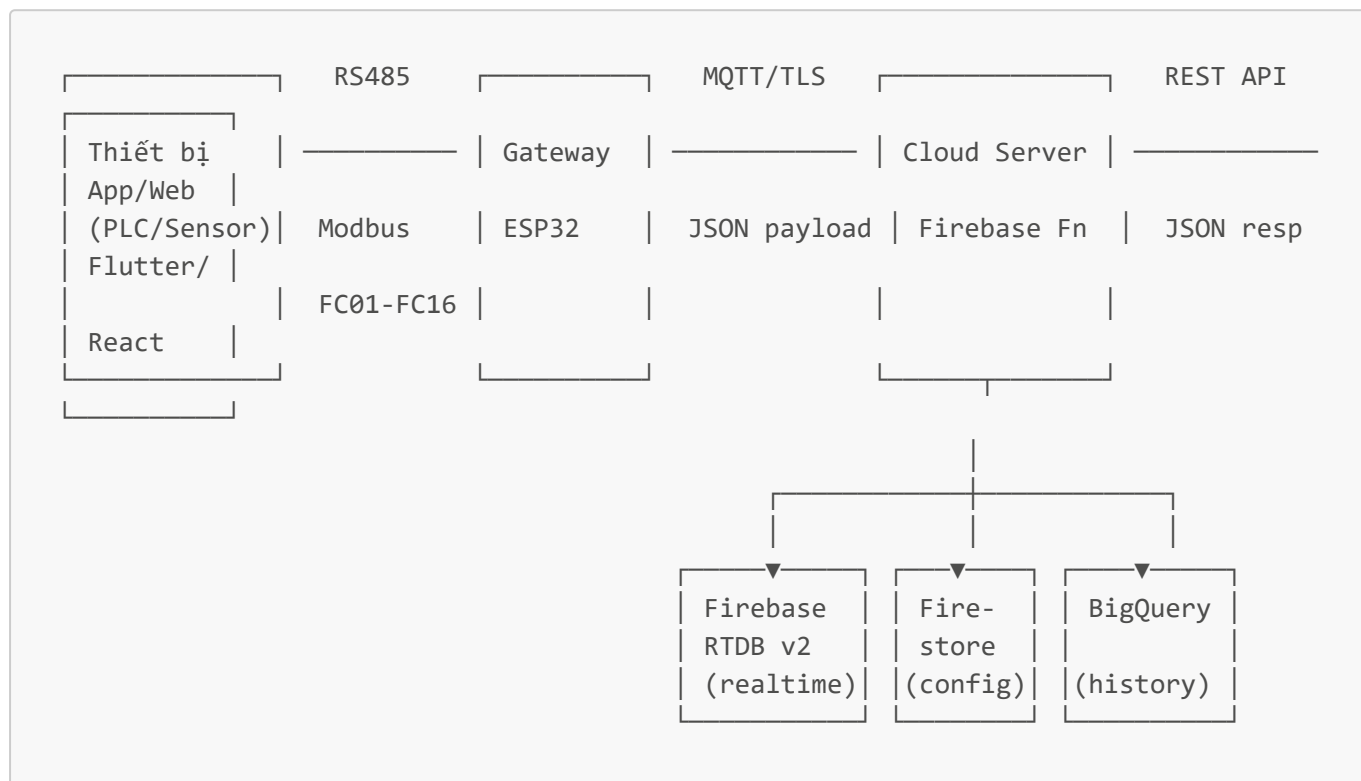
6. DỰ ÁN ĐÃ TRIỂN KHAI

#	Dự án	Ngành	Mô tả
1	Nhà máy nước	Xử lý nước	Giám sát lưu lượng, áp suất, chất lượng nước
2	LIXIL Vietnam	Sản xuất	Giám sát năng lượng nhà máy
3	VSIP Industrial Park	Khu công nghiệp	Giám sát hạ tầng KCN
4	Poongin Vina	Sản xuất	Giám sát thiết bị sản xuất
5	PTT Food	Thực phẩm	Giám sát nhiệt độ kho lạnh
6	Hệ thống Gas	Năng lượng	Giám sát áp suất, lưu lượng gas

#	Dự án	Ngành	Mô tả
7	Hệ thống PCCC	Phòng cháy	Giám sát trạng thái hệ thống PCCC
8	Phòng Server	Data Center	Giám sát nhiệt độ, độ ẩm, UPS

7. LUỒNG DỮ LIỆU CHI TIẾT (Data Flow)

7.1 Tổng quan luồng dữ liệu



7.2 Firmware → Cloud: Gửi dữ liệu lên server

Bước 1: Gateway đọc Modbus

Gateway ESP32 chạy vòng lặp mỗi 1-5 giây:
 custom_modbus.c → RS485 TX/RX → đọc FC03/FC04 → parse INT16/UINT32/FLOAT32
 → Lưu vào bộ nhớ RAM → chuẩn bị JSON payload

Bước 2: Gateway gửi MQTT lên Cloud

mqtt.c / iot.c → JSON payload → MQTT + TLS → ClearBlade IoT Core → Cloud Function

Format V1 (full — firmware cũ):


```

{
  "Location": "STATION-001",
  "LastTime": "2026-06-05 15:30:00",
  "WifiInfo": "WiFi -45dBm",
  "RS485Data": [
    {
      "Index": 0,
      "Name": "Temp",
      "GroupName": "SENSORS",
      "MemoryType": 1,
      "Value": 25.5,
      "Type": "float",
      "Unit": "°C",
      "Address": 100,
      "SlaveId": 1,
      "IsCycleTime": true
    },
    {
      "Index": 1,
      "Name": "Pump1",
      "GroupName": "CONTROL",
      "MemoryType": 3,
      "CoilValue": true,
      "Type": "coil"
    }
  ]
}

```

Format V2 (compact — firmware mới, tiết kiệm bandwidth):

```

{
  "Location": "STATION-001",
  "LastTime": "2026-06-05 15:30:00",
  "V": 2,
  "RS485Data": [
    { "I": 0, "V": 25.5, "C": false, "CT": true },
    { "I": 1, "V": 1, "C": true, "CT": true }
  ]
}

```

Server dùng `enrich-v2.js` để chuyển V2 → V1 format, tra cứu metadata (Name, GroupName, MemoryType...) từ RTDB cache (TTL 2 phút, max 500 devices).

Bước 3: Cloud Function xử lý

3 API endpoints nhận data:

Endpoint	Mô tả
POST /api/handleRS485Data2	Chính — đầy đủ: RTDB + BigQuery + Alarm + Automation
POST /api/handleRS485DataNotSaveBigquery	Nhẹ — bỏ BigQuery (cho high-frequency updates)
POST /api/a	Legacy v1 (dùng hàm cũ, không tối ưu)

Flow xử lý **handleRS485Data2** (fire-and-forget):

```

POST /api/handleRS485Data2 (nhận data từ gateway)
├── 0. Response HTTP 200 ngay lập tức (không chờ xử lý xong)
│   └── ★ Fire-and-forget: gateway không cần chờ → giảm timeout
├── 1. Enrich data (nếu V2 → chuyển về V1 format)
│   └── enrich-v2.js: tra metadata từ RTDB cache (TTL 2 phút)
├── 2. updateRealTimeFirebase2() – Ghi RTDB v2
│   ├── Path: /Devices/DAIVIET-RS485/{Location}/RS485Data/{Index}/
│   ├── Ghi: Value, CoilValue, LastTime, WifiInfo
│   ├── ★ Detect CoilValue changes (before vs after → trả về coilChanges[])
│   └── ★ Tối ưu: sensor-only → 0 RTDB reads; coil → chỉ read node coil
├── 3. SaveSensorsDataToBigQueryOrSetting() – Lưu lịch sử
│   ├── MemoryType=1 (SENSOR): → BigQuery SENSOR_DATASET.SENSOR_TABLE
│   │   └── { DeviceId, SensorId, Value, Timestamp, Status }
│   ├── MemoryType=2 (SETTING): → Firestore SensorSettings/{Location}/{Group}
│   └── MemoryType=3 (COIL): → Firestore CoilSettings/{Location}/{Group}
├── 4. HandleAlarms2() – Kiểm tra ngưỡng cảnh báo
│   └── So sánh Value với HighAlarm/LowAlarm → xem chi tiết bên dưới
├── 5. processCoilChanges(coilChanges) – Xử lý alarm cho coil
│   └── Nếu CoilValue thay đổi → tạo Cloud Task delay 30s
└── 6. checkAutomationConditions() – Kiểm tra automation rules
    └── Data Forward, sensor conditions → xem mục 7.9

```

7.3 Lưu trữ dữ liệu — 3 tầng

Tầng	Database	Mục đích	Dữ liệu	Thời gian giữ
Realtime	Firebase RTDB v2	Hiển thị trên App/Web	Giá trị hiện tại	Ghi đè liên tục
Config	Cloud Firestore	Cấu hình, người dùng	Settings, Alarms, Users	Vĩnh viễn

Tầng	Database	Mục đích	Dữ liệu	Thời gian giữ
Lịch sử	Google BigQuery	Biểu đồ, báo cáo, thống kê	Toàn bộ sensor data	Dài hạn (3-5 năm+)

Firebase RTDB v2 — Cấu trúc dữ liệu realtime

```

/Devices/DAIVIET-RS485/
├── {Location}/                                # VD: STATION-001
│   ├── LastTime: "2026-06-05 15:30:00"
│   ├── LastIndex: 5
│   ├── WifiInfo: "WiFi -45dBm"
│   └── RS485Data/
│       ├── 0/                                # Index 0
│       │   ├── Name: "Temp"
│       │   ├── GroupName: "SENSORS"
│       │   ├── MemoryType: 1
│       │   ├── Value: 25.5                    # ← App/Web đọc realtime từ đây
│       │   ├── Unit: "°C"
│       │   └── IsHighAlarm: false
│       ├── 1/                                # Index 1
│       │   ├── Name: "Pump1"
│       │   ├── CoilValue: true                # ← Trạng thái ON/OFF
│       │   └── MemoryType: 3
│       └── ...
└── ...

```

BigQuery — Schema bảng sensor

```

-- SENSOR_DATASET.SENSOR_TABLE (lịch sử sensor)
DeviceId  STRING    -- "STATION-001"
SensorId  STRING    -- "Temp"
Value     FLOAT64   -- 25.5
Timestamp INTEGER   -- Unix timestamp (seconds)
Status    INTEGER   -- 0=normal, 1=calibration, 2=error

-- NOTI_DATASET.ALARM_TABLE (lịch sử alarm)
LocationId STRING   -- "STATION-001"
GroupName  STRING   -- "SENSORS"
DeviceName STRING   -- "Temp"
Content    STRING   -- "15:30:00: Temp vượt ngưỡng cao: 45.2°C > 40°C"
Time       INTEGER   -- Unix timestamp

```

Firestore — Collections chính

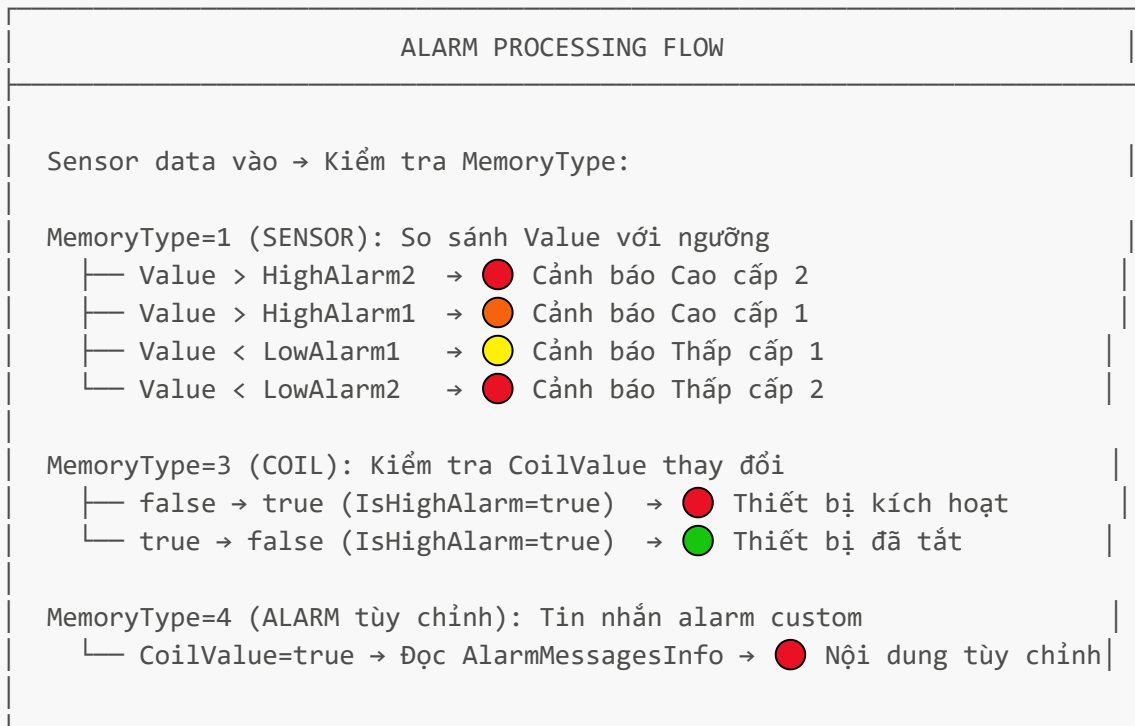
```

ListDevices/RS485                                # Tên hiển thị: { "STATION-001": "Nhà máy Kiên
Giang" }

```

SensorSettings/{Location}/{Group}	# Cài đặt ngưỡng (HighAlarm, LowAlarm...)
CoilSettings/{Location}/{Group}	# Trạng thái coil (ON/OFF)
AlarmMessagesInfo/{Location}	# Nội dung alarm custom
AlarmConfig/{Location}	# Config: delay, repeat, channels
Users/{userId}	# Thông tin user, quyền

7.4 Hệ thống Cảnh báo (Alarm System)



Delay chống báo giả (Cloud Tasks)

- Sensor vượt ngưỡng lần đầu
 - Tạo Cloud Task với delay (VD: 30 giây)
 - Task name: alarm-{location}-{sensorName}-{timestamp}
 - Queue: alarm-delay-queue (asia-southeast1)
- Trong 30 giây:
 - Giá trị trở về bình thường → Xóa Cloud Task → Không báo
 - Vẫn vượt ngưỡng sau 30s → Cloud Task fire → Gửi alarm
- Alarm gửi qua 4 kênh song song:
 - └ FCM Push Notification → App iOS/Android/Web
 - └ Email (SendGrid/Nodemailer) → Danh sách email
 - └ Zalo ZNS (node-zalo-bot) → Số Zalo đăng ký
 - └ BigQuery log → NOTI_DATASET.ALARM_TABLE

Alarm Repeat (nhắc lại nếu chưa xử lý)

Nếu alarm chưa acknowledged:

- Gửi lại sau X phút (configurable)
- Tối đa N lần nhắc (configurable)
- Cho đến khi user bấm "Đã xử lý" (acknowledgeAlarm)

Alarm cho PCCC (Phòng cháy)

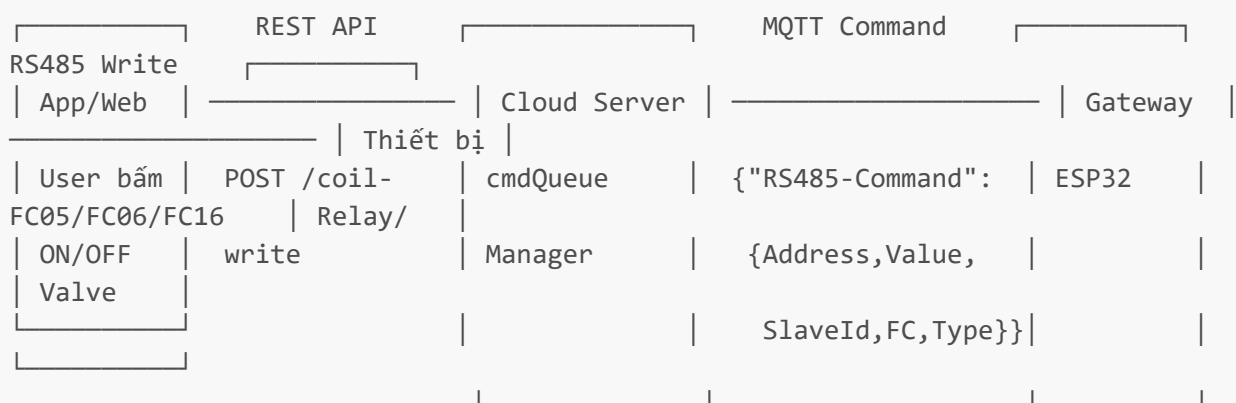
Dự án có tên chứa "pccc":

- Sound: alarm.caf (âm báo cháy khẩn cấp)
- Priority: HIGH (apns-priority: 10)

Dự án khác:

- Sound: success.caf (âm thông báo thường)

7.5 Điều khiển từ xa (Remote Control)



Chi tiết cmdQueueManager (gom lệnh 100ms):

1. User bấm ON → API: POST /coil-write { deviceId, commands: [...] }
2. cmdQueueManager.enqueue(deviceId, commands)
 - Buffer 100ms (gom nhiều lệnh trong 100ms thành 1 batch)
 - Ưu tiên: bool/coil > int > int32 > float
 - Last-value-wins nếu trùng Name
3. Sau 100ms → Gửi 1 MQTT duy nhất: {"RS485-Commands": [...]}
4. Gateway nhận → Modbus write FC05/FC06/FC16 → Thiết bị thực thi
5. Gateway đọc lại giá trị → Gửi confirm lên Cloud → App/Web cập nhật

MQTT Command Format:

```
{
  "RS485-Command": {
    "Address": 100,
```

```

    "SlaveId": 1,
    "FunctionCode": 5,
    "DataLength": 1,
    "Type": "coil",
    "Name": "Pump1",
    "Value": true
  }
}

```

7.6 App/Web đọc dữ liệu

Realtime (Dashboard)

App Flutter / Web React

- Firebase SDK listen: /Devices/DAIVIET-RS485/{Location}/RS485Data/
- onValue callback → cập nhật UI tức thì
- Không cần polling – Firebase push tự động khi data thay đổi

Biểu đồ lịch sử

App/Web → REST API: GET /chart-data?deviceId=X&sensor=Temp&from=...&to=...

- Cloud Function → BigQuery query:


```
SELECT Timestamp, Value FROM SENSOR_TABLE
WHERE DeviceId=@id AND SensorId=@sensor AND Timestamp BETWEEN @from AND @to
```
- Response JSON → Highcharts/Syncfusion render

Bản đồ

App/Web → Firestore: ListDevices/RS485 (tên trạm)

- + RTDB: /Devices/DAIVIET-RS485/{Location}/LastTime (online/offline)
- + Firestore: Location coordinates
- Google Maps render markers (● online / ● alarm / ● offline)

7.7 Báo cáo (Report System)

Trigger: Tự động (daily/weekly/monthly) hoặc User yêu cầu

- 1. Query BigQuery (report-queries.js – 15 loại query)
 - queryRawData – Dữ liệu thô
 - queryAvg24h – Trung bình 24h
 - queryAvg1hMonth – TB 1h theo tháng
 - queryAvg1hMaxMonth – Max 1h theo tháng
 - queryAvg1hMinMonth – Min 1h theo tháng

- └─ queryDataRateRange – Tỷ lệ dữ liệu (uptime)
- └─ queryForQCVNExceedance – Vi phạm QCVN
- └─ queryThresholdExceedCounts – Đếm vượt ngưỡng
- └─ ...9 loại theo TT 10/2021/TT-BTNMT
- ─ 2. Tạo Excel (excel-v2.js)
 - └─ Header: Logo công ty, tên, địa chỉ, SĐT, email
 - └─ Data table: Conditional formatting (đỏ nếu vượt ngưỡng)
 - └─ Summary: Min/Max/Avg/Count
 - └─ Biểu đồ nhúng: ChartJS server-side render → PNG → embed
 - └─ Status column: 00=normal, 01=calibration, 02=error
- ─ 3. Upload Cloud Storage
- ─ 4. Gửi email tự động (SendGrid/Nodemailer)
 - └─ Đính kèm file Excel/PDF

Function `httpexportexcel` dùng 1GB RAM — vì báo cáo lớn cần nhiều bộ nhớ cho query BigQuery + render chart + tạo Excel

7.8 Realtime via Socket.IO

- navis-socket-io/ (Port 1881):
- └─ PowerMeter ↔ Mobile: Bridge dữ liệu WPF desktop → mobile viewers
 - └─ QR Login: Web hiện QR → App scan → Socket confirm → Web auto-login
 - └─ Alarm relay: Server push alarm → Socket.IO → Web notification
 - └─ Video relay: RTSP camera → Socket.IO → Web player

7.9 Automation System (Tự động hóa)

- Mỗi lần nhận data từ gateway → `checkAutomationConditions(location, RS485Data)`
- └─ Đọc rules từ Firestore: Automations/{deviceId}/rules (cached 5 phút, max 1000 devices)
 - └─ Rule type: sensor_condition
 - └─ Giá trị sensor đạt điều kiện → thực thi action (VD: bật/tắt relay)
 - └─ Rule type: data_forward (quan trọng nhất)
 - └─ Trạm A gửi data → Cloud tự động chuyển tiếp sang Trạm B
 - └─ Change detection: so sánh scan hiện tại vs scan trước (in-memory)
 - └─ Analog throttle: 5000ms giữa các lần gửi float
 - └─ Bool: gửi ngay khi thay đổi
 - └─ Boot sync: POST /api/requestSync – đồng bộ toàn bộ khi trạm B khởi động

7.10 Cấu hình thiết bị từ xa (Device Config)

App/Web → POST /api/pushConfigPageV2

- 1. Config lớn → chia thành pages (ESP32 bộ nhớ giới hạn)
Mỗi page gửi riêng → lưu tạm RTDB: /ConfigStaging/{deviceId}/pages/{n}
- 2. Page cuối (isFinal=true):
 - Gom tất cả pages → full config
 - Lưu full config vào RTDB: /Devices/DAIVIET-RS485/{deviceId}/RS485Data/
 - buildMinimalConfig() → strip Name, GroupName, Unit → chỉ giữ Modbus params (thu nhỏ ~10x → fit vào bộ nhớ ESP32)
 - Push minimal config → ClearBlade IoT → MQTT → Gateway
 - Invalidate enrich cache (metadata đã thay đổi)
- 3. Gateway nhận config mới → restart Modbus polling với config mới

7.11 Tổng hợp API Endpoints

Method	Route	Mô tả
POST	/api/handleRS485Data2	Nhận data từ gateway (chính)
POST	/api/handleRS485DataNotSaveBigquery	Nhận data (không lưu BigQuery)
POST	/api/handleCoilDevice	Điều khiển ON/OFF từ App
POST	/api/send-command	Gửi lệnh MQTT trực tiếp
POST	/api/pushConfigPageV2	Push cấu hình Modbus
POST	/api/getListDevices	Danh sách trạm của user
POST	/api/excel	Tạo & email báo cáo Excel
GET	/api/excel-for-mobile	Download báo cáo cho App
GET	/api/excel-for-web	Download báo cáo cho Web
POST	/api/excel-alarm	Báo cáo lịch sử cảnh báo
POST	/subscribe-to-topic	Đăng ký nhận push notification
POST	/bulk-topic-action	Đăng ký/hủy hàng loạt
POST	/api/checkCoilAndNotify	Cloud Task: kiểm tra coil alarm
POST	/api/handleAlarmDelay	Cloud Task: alarm delay
POST	/api/requestSync	Boot sync cho DataForward
POST	/api/automation/execute	Cloud Task: automation rule
GET	/noti-data	Lịch sử cảnh báo (BigQuery)
POST	/api/report-ai-summary	Báo cáo AI tóm tắt tuần
POST	/api/analyze-devices	Phân tích sức khỏe thiết bị

Method	Route	Mô tả
POST	/api/update-device-config	Cập nhật config thiết bị

8. QUY TRÌNH TRIỂN KHAI CHO KHÁCH HÀNG

Bước 1: CHUẨN BỊ THIẾT BỊ

- | Tủ điện có RS485 Modbus, cảm biến, PLC...
- | Xác định danh sách tag cần giám sát



Bước 2: LẮP ĐẶT GATEWAY

- | Đấu dây RS485 từ Gateway → thiết bị
- | Cấp nguồn 12-24VDC cho Gateway



Bước 3: CẤU HÌNH TRÊN APP

- | Tải App → Quét QR Gateway
- | Cài đặt: Modbus Address, Baud Rate, Data Type
- | Kết nối WiFi/LAN/4G



Bước 4: KIỂM TRA & HIỆU CHỈNH

- | Xác nhận dữ liệu đúng trên Dashboard
- | Cài đặt ngưỡng cảnh báo
- | Phân quyền người dùng



Bước 5: BÀN GIAO & VẬN HÀNH

- | Hướng dẫn sử dụng cho khách
- | Bàn giao tài liệu
- | Hỗ trợ kỹ thuật từ xa

9. LIÊN HỆ

Kênh	Thông tin
Liên hệ kỹ thuật	Mr. Cường — 0932.786.579
Hotline	0286 657 2981
Zalo	0286 657 2981
Email	omwater.vn@gmail.com / info@iotdaiviet.com
Website	navis.iotdaiviet.com
Công ty	Đại Việt IoT Technology JSC
Đối tác	O&M Water
Địa chỉ	TP. Hồ Chí Minh, Việt Nam
Zalo OA ID	2255886536825866266

10. 🕒 TIMELINE TRIỂN KHAI & HỖ TRỢ

Thời gian triển khai: 1-2 tuần/trạm

Tuần	Công việc
Tuần 1	Khảo sát, lắp đặt, kết nối, cấu hình
Tuần 2	Cài cảnh báo, báo cáo, training, nghiệm thu

SLA hỗ trợ kỹ thuật

Mức độ	Thời gian phản hồi
🔴 Nghiêm trọng	1-2 giờ
🟡 Bình thường	2-4 giờ (ngày làm việc)
🟢 Yêu cầu hỗ trợ	24 giờ

Nghiệm thu đạt khi:

- ✅ Gateway kết nối ổn định, dữ liệu realtime trên App/Web
- ✅ Tất cả tag đọc đúng giá trị
- ✅ Cảnh báo ngưỡng hoạt động chính xác
- ✅ Biểu đồ và xuất báo cáo hoạt động
- ✅ Phân quyền và chia sẻ QR hoạt động
- ✅ Đào tạo nhân viên hoàn tất
- ✅ Bàn giao tài liệu

11. 📝 GHI CHÚ CHO AI ASSISTANT

Khi đọc tài liệu này, lưu ý:

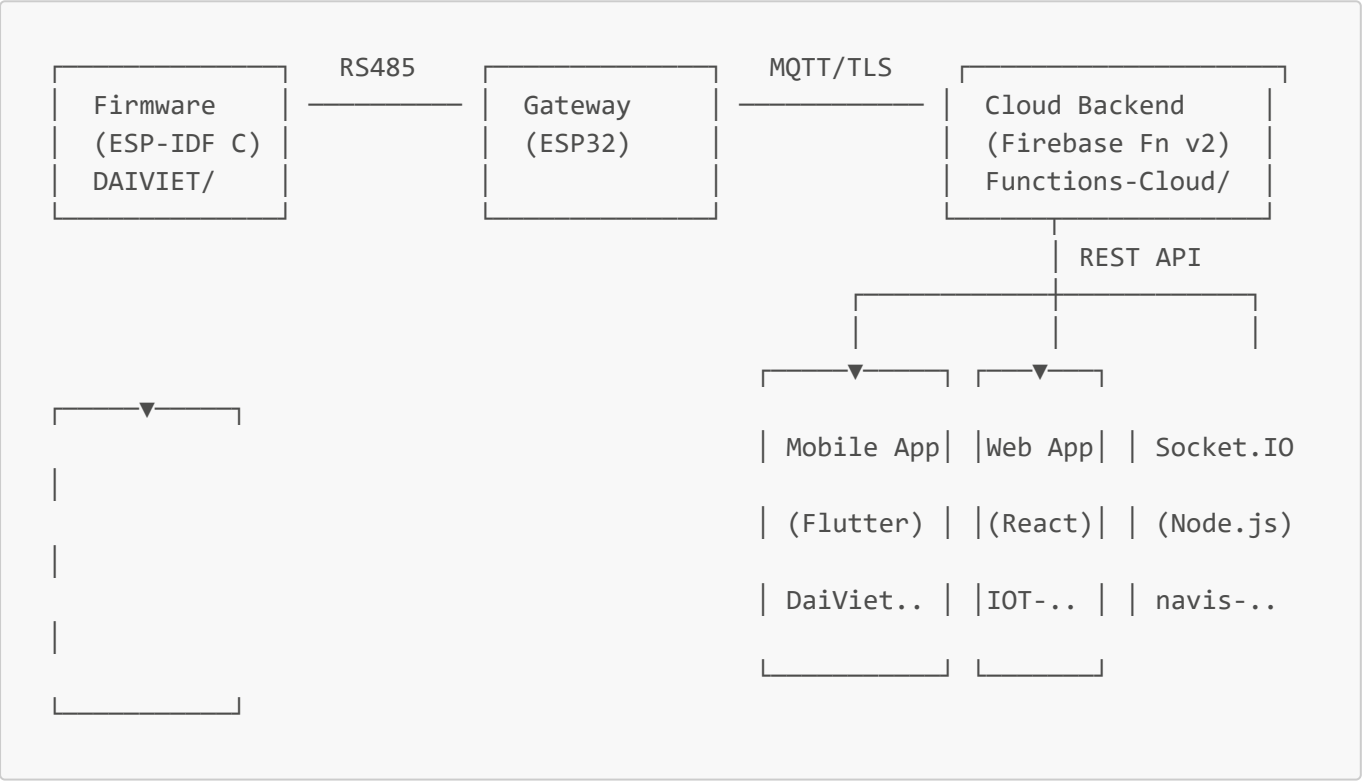
- Navis MOD** = tên thiết bị gateway phần cứng (không phải phần mềm)
- Modbus RTU** = giao thức công nghiệp phổ biến nhất, dùng dây RS485
- Tag** = 1 điểm dữ liệu (ví dụ: nhiệt độ nước = 1 tag)
- Trạm** = 1 địa điểm lắp đặt (có thể có nhiều gateway)
- MQTT** = giao thức realtime nhẹ, phù hợp IoT
- P&ID** = Process & Instrumentation Diagram (sơ đồ công nghệ nhà máy)
- OTA** = Over-The-Air update (cập nhật firmware gateway từ xa)
- Website là **static site** (HTML/CSS/JS thuần), host trên **GitHub Pages**
- Tên miền navis.iotdaiviet.com trỏ qua CNAME
- Slide dùng scroll-snap fullscreen, không dùng framework slide nào

12. 🔧 SOURCE CODE — Dành cho kỹ thuật nội bộ

⚠️ **PHẦN NÀY CHỈ DÀNH CHO KỸ THUẬT NỘI BỘ CÔNG TY** Không chia sẻ ra ngoài. Chứa đường dẫn code, tech stack, kiến trúc hệ thống.

12.1 Tổng quan 4 Repository

#	Repo	Đường dẫn	Tech Stack	Mô tả
1	Firmware	C:\Users\Tri\DAIVIET	ESP-IDF (C) + Google IoT SDK	Code chạy trên Gateway ESP32
2	Mobile App	C:\Users\Tri\Mobile-App\DaiVietIoTMobile	Flutter 2.x (Dart)	App iOS + Android
3	Web App	C:\Users\Tri\IOT-Web\IOT-Monitoring	React 18 + MUI + Highcharts	Web dashboard + services
4	Cloud Backend	C:\Users\Tri\Functions-Cloud-v2	Firebase Functions (Node 22)	API + triggers + reports



12.2 FIRMWARE — C:\Users\Tri\DAIVIET

Tech Stack

Component	Technology
MCU	ESP32 (Espressif)
Framework	ESP-IDF (Espressif IoT Development Framework)
Ngôn ngữ	C
Build System	CMake
Cloud SDK	Google IoT Device SDK Embedded C (iot-device-sdk-embedded-c)

Component	Technology
TLS	mbedtls (tích hợp sẵn ESP-IDF)
Network	LWIP stack, hỗ trợ PPP (cho 4G)
Protocol	MQTT qua Google Cloud IoT Core

Cấu trúc thư mục

```

DAIVIET/
├── CMakeLists.txt           # CMake build – link Google IoT SDK + mbedtls
├── Kconfig                 # Menu config: Project ID, Location, Registry
ID, Device ID
├── sdkconfig.defaults      # Mặc định: bật PPP/4G, LWIP stack 4096
├── component.mk            # Legacy component makefile
├── README.md              # "ESP-Google-IoT" + "MODBUS-FIRMWARE-V2"
├──
├── source/                # 🔥 SOURCE CODE CHÍNH
│   ├── rs485_module/      # ★ MAIN FIRMWARE PROJECT
│   │   ├── main/
│   │   │   ├── main.c     # app_main() entry point (488 dòng)
│   │   │   └── modbus/    # custom_modbus.c (284KB! – đọc/ghi Modbus
RTU)
│   │   ├── network/      # Tất cả kết nối:
│   │   │   ├── ethernet.c # LAN/RJ45 DHCP
│   │   │   ├── wifi.c    # WiFi + SmartConfig + WPS
│   │   │   ├── mqtt.c    # MQTT client
│   │   │   ├── iot.c     # Google IoT Core / ClearBlade
│   │   │   ├── BLE_Client.c # Bluetooth → HUMATIC sensors
│   │   │   ├── esp_ota.c  # OTA firmware update
│   │   │   └── ntp.c      # Time sync
│   │   ├── storage/      # NVS flash storage
│   │   ├── config/       # Device config
│   │   ├── components/   # ESP-IDF components:
│   │   │   ├── esp32-wifi-manager/ # WiFi AP + portal
│   │   │   ├── espressif_esp_modem/ # 4G modem driver (v0.1.15)
│   │   │   ├── esp-double-reset/   # Double reset detection
│   │   │   ├── esp-status-led/     # LED patterns
│   │   │   ├── esp-wifi-reconnect/ # Auto reconnect
│   │   │   ├── esp-wps-config/     # WPS support
│   │   │   ├── context/            # App context struct
│   │   │   ├── error/              # Error handling
│   │   │   └── utils/              # Helpers
│   │   ├── firebase.json          # Service account (project:
weatherstationiotdaiviet)
│   │   ├── partitions.csv         # Flash partition table
│   │   ├── sdkconfig              # Full SDK configuration
│   │   └── sim_debug_tool/        # SIM debugging tool (separate IDF project)
├── iot-device-sdk-embedded-c/    # Google Cloud IoT SDK (submodule)
├── port/                          # POSIX BSP port (net, mem, rng, time)

```

```

├── tools/modbus_slip_simulator/    # Modbus simulator tool
├── build/                          # Build output (.bin)
├── *.json                         # Config JSON cho từng dự án:
│   ├── kien_giang_plc_config.json #   Kiên Giang (70KB, nhiều tag)
│   ├── nuocxanh.json              #   Nước Xanh (87KB)
│   └── flow_converter_config.json  #   Flow converter
└── cleanup.py, generate_test_config.js # Build scripts

```

Boot Sequence (main.c)

```

1. NVS init → 2. Watchdog (600s timeout)
3. GPIO init → 4. App Context init
5. Storage init → 6. Network (ETH/WiFi/4G tùy config)
7. OTA check → 8. NTP time sync
9. Chờ network + time (timeout 60s)
10. IoT/MQTT connect → 11. Modbus polling start
12. Health monitor task (check RAM <5KB → restart, MQTT timeout 90min → restart)

```

Build & Flash

```

# Cài đặt ESP-IDF v3.2+ trước
# https://docs.espressif.com/projects/esp-idf/en/latest/get-started/

# Build
idf.py build

# Flash qua USB
idf.py -p COM3 flash monitor

# Config
idf.py menuconfig    # → Google IoT Core Configuration

```

Data Flow (trong firmware)

```

1. Modbus Poll Loop (mỗi 1-5 giây):
   modbus_master.c → RS485 TX/RX → đọc FC03/FC04 → parse INT16/UINT32/FLOAT32

2. MQTT Publish (mỗi 5-30 giây):
   mqtt_handler.c → JSON payload → MQTT + TLS → Google Cloud IoT Core

3. MQTT Subscribe (lắng nghe lệnh):
   Cloud → MQTT command → mqtt_handler.c → modbus_master.c → ghi FC05/FC06/FC16

```

4. OTA Update:

Cloud → MQTT OTA command → ota_update.c → download .bin → flash → reboot

12.3 MOBILE APP — C:\Users\Tri\Mobile-App\DaiVietIoTMobile

Tech Stack

Component	Technology
Framework	Flutter 2.x (SDK >=2.12.0 ❤️ .0.0)
Ngôn ngữ	Dart
App Name	rochauhoa (package name nội bộ)
Version	1.0.601
State Management	Provider / Bloc (tùy module)
Backend	Firebase Realtime Database
Auth	Firebase Auth (Email, Google, Apple, Phone OTP)
Push	Firebase Cloud Messaging (FCM)
Charts	Syncfusion Flutter Charts
Maps	Google Maps Flutter
Navigation	Curved Navigation Bar
Fonts	SF UI Display (custom)

State Management

Provider (v6.0.2) + GetX (v4.6.6) + RxDart (v0.27.1)

Dependencies chính (pubspec.yaml — 150+ packages)

```
# Firebase
firebase_core 3.3.0, firebase_auth 5.1.1, cloud_firestore 5.2.1
firebase_database 11.0.1, firebase_messaging 15.0.4, firebase_storage 12.1.3
firebase_crashlytics, firebase_app_check, firebase_dynamic_links
cloud_functions 5.0.4

# Charts & Gauges
syncfusion_flutter_charts 21.2.10, fl_chart 0.36.3
syncfusion_flutter_gauges 21.2.10

# Maps
google_maps_flutter 2.4.2, map_location_picker (custom git)
```

```
# Video/RTSP
media_kit 1.1.10, flutter_vlc_player 7.4.2, camera 0.10.5+9

# BLE (kết nối HUMATIC handheld)
flutter_blue_plus 1.32.5

# Socket.IO (realtime)
socket_io_client 1.0.1

# WiFi Config (cấu hình ESP32)
esptouch_smartconfig 0.0.6, wifi_iot 0.3.17

# Auth
google_sign_in 6.2.1, sign_in_with_apple 6.1.4

# QR/Barcode
flutter_barcode_scanner, qr_flutter, flutter_qr_reader, zxing2

# UI
flutter_zoom_drawer, curved_navigation_bar, glassmorphism
google_fonts, lottie, flutter_spinkit, shimmer, cached_network_image
```

Cấu trúc thư mục

```
DaiVietIoTMobile/
├── pubspec.yaml                # Dependencies + version 1.0.601
├── firebase.json              # Firebase config
├── GoogleService-Info.plist   # iOS Firebase config
├── lib/                       # 🔥 DART SOURCE CODE
│   ├── main.dart              # Entry point (250KB – monolithic)
│   ├── MVC/                   # MVC architecture:
│   │   ├── Common/           # Shared code
│   │   ├── Controllers/      # Business logic
│   │   ├── Dialogs/          # Dialog widgets
│   │   ├── Models/           # Data models
│   │   ├── Pages/            # MVC page views
│   │   ├── Services/         # API services
│   │   └── Widgets/          # Reusable widgets
│   ├── pages/                 # ★ Feature pages (files lớn):
│   │   ├── ProjectPage.dart  # (582KB!) – Quản lý dự án/trạm
│   │   ├── settingPage.dart  # (239KB) – Cài đặt Modbus, ngưỡng
│   │   ├── notiPage.dart     # (87KB) – Thông báo/cảnh báo
│   │   ├── detailPage.dart   # (69KB) – Chi tiết sensor
│   │   ├── homePage.dart     # (51KB) – Dashboard tổng quan
│   │   ├── AutomationPage.dart # Tự động hóa
│   │   ├── powerMeterPage.dart # Đồng hồ điện
│   │   └── TedcoPage.dart    # Tedco integration
│   └── model/                 # Data models:
│       ├── RS485/            # Modbus data models
│       └── SmarthHome/       # Smart home models
```


- └─ MyChart/, DetailChart/ # Highcharts wrapper
- └─ CardValueSensor/ # Sensor card realtime
- └─ CoilValueDevice/ # ON/OFF control card
- └─ GoogleMapSimple/, MapD/ # Map components
- └─ SmartCameraPlayer/ # RTSP player
- └─ ExcelExportButton/ # Export Excel
- └─ CNVDIALOG*/ # QCVN standards dialog
- └─ FCMNotification/ # Push notification handler
- └─ WifiSignal/, QuickView/ # Widgets
- └─ redux/store.js + reducer/ # Redux state
- └─ contexts/ThemeContext.js # Dark/light mode
- └─ hooks/useListDevice.js # Custom hooks
- └─ config/firebase.js # Firebase init (project: weatherstationiotdaiviet)
- └─ scss/ # SASS styles
- └─ navis-socket-io/ # 🖱️ Socket.IO Server (Node.js)
 - └─ index.js # Port 1881 – Express + Socket.IO + FCM
 - └─ package.json # socket.io 2.3.0, firebase-admin 12
- └─ ftp-server/ # 📁 FTP Server (Python)
 - └─ ftp_server.py # FTP daemon port 21
 - └─ web_dashboard.py # Dashboard port 8080 (109KB!)
 - └─ firebase_handler.py # → Firebase RTDB
 - └─ bigquery_logger.py # → BigQuery
 - └─ alarm_handler.py # Alarm processing
 - └─ email_sender.py # Email alerts
 - └─ parser.py, config.py # File parser + config
- └─ humatic-web/ # 🏠 Humatic Web (white-label React app)
- └─ domain-manager/ # 🌐 Domain + SSL manager (port 9090)
- └─ rtsp-mp4/ # 📺 RTSP to MP4 converter
- └─ deployment/ # 🚀 Deploy scripts
 - └─ deploy-all.sh # Master deploy
 - └─ server-setup.sh # VM setup (Node, Nginx, PM2, Certbot)
 - └─ iot-monitoring.conf # Nginx config
 - └─ navis-socket.conf # Socket.IO proxy
 - └─ humatic-web.conf # Humatic config
- └─ docker-compose.yml # 🐳 Docker (7 services)
- └─ Dockerfile, Dockerfile.ftp, Dockerfile.node
- └─ .github/workflows/ # CI/CD pipelines

Docker Services

Service	Container	Port	Mô tả
nginx	iot-nginx	80, 443	Reverse proxy + SSL
iot-monitoring-build	iot-monitoring-build	—	React build → static files

Service	Container	Port	Mô tả
humatic-web-build	humatic-web-build	—	Humatic React build
ftp-server	iot-ftp	21, 8080	FTP + dashboard
navis-socket-io	iot-navis-socket	1881	Socket.IO realtime
domain-manager	iot-domain-manager	9090	Domain + SSL manager
certbot	iot-certbot	—	Auto SSL renewal

Build & Deploy

```
# Dev (local)
npm start                                # React dev server :3000

# Production build
npm run build                            # → build/ folder

# Docker deploy
docker-compose up -d                    # Start all services
docker-compose up -d --build            # Rebuild + start
docker-compose logs -f                  # View logs

# SSL
sudo certbot --nginx -d yourdomain.com
```

12.5 CLOUD BACKEND — C:\Users\Tri\Functions-Cloud-v2

Tech Stack

Component	Technology
Platform	Firebase Cloud Functions v2
Runtime	Node.js 22
Framework	Express.js
Database	Firebase Realtime Database + Cloud Firestore
Data Warehouse	Google BigQuery
Storage	Google Cloud Storage
Auth	Firebase Auth + JWT (jsonwebtoken)
IoT	Google Cloud IoT Core / ClearBlade IoT
Email	SendGrid + Nodemailer
Notifications	Firebase Cloud Messaging (FCM)

Component	Technology
Reports	ExcelJS + ChartJS-Node-Canvas + xlsx-chart
Zalo	node-zalo-bot (ZNS notifications)
Entry Point	<code>index-v2.js</code> (modular, thay thế file gốc 17,893 dòng)

Firebase Project: `weatherstationiotdaiviet`

Exported Functions (4 functions)

Function	Version	Region	RAM	Concurrency	Max Instances
<code>HttpPostRequestV2</code>	v2	asia-east2	256MB	5	3
<code>HttpPostRequestV3</code>	v2	us-central1	256MB	30	15
<code>httpexportexcel</code>	v2	us-central1	1GB	—	—
<code>HttpPostRequest</code>	v1 (legacy)	asia-east2	256MB	—	—

Databases (4 databases)

Database	Type	URL/ID
<code>dbStore</code>	Firestore	(primary)
<code>db</code>	Realtime DB v1	<code>weatherstationiotdaiviet.firebaseio.com</code>
<code>dbV2</code>	Realtime DB v2	<code>iotdaiviet-realtime-db.asia-southeast1.firebaseio.com</code>
BigQuery	Data Warehouse	dataset: <code>weather_station_iot</code> , table: <code>raw_data</code>

Cấu trúc thư mục

Functions-Cloud-v2/	
├─ index-v2.js	# 🔥 ENTRY POINT – Express app, 4 exported
functions	
├─ package.json	# Dependencies (Node 22)
├─ firebase.json	# Hosting + functions config
├─ .firebaserc	# Project aliases
├─ .runtimeconfig.json	# Runtime config (Firebase + BigQuery)
├─ .config.json	# BigQuery dataset/table config
├─ key.json	# GCP service account key
├─ routes/	# 🔥 API ROUTES
│ └─ all-routes.js	# 491KB! – 109 routes (refactored từ index.js
17,893 dòng)	
│ └─ report-routes.js	# 108KB – report generation
│ └─ activity-routes.js	# 8KB – activity logging
└─ helpers/	# 🧩 BUSINESS LOGIC

alarms.js	# 59KB – alarm processing + multi-level thresholds
excel-v2.js	# 36KB – Excel export (ExcelJS)
report-queries.js	# 19KB – report data queries
enrich-v2.js	# 17KB – data enrichment
report-excel.js	# 15KB – Excel report builder
qcvn-defaults.js	# 12KB – QCVN environmental standards
firebase-rt.js	# 8KB – Realtime DB helpers
api-stats.js	# 6KB – API statistics
bigquery.js	# 5.5KB – BigQuery insert/query
iot-commands.js	# 5KB – IoT device commands (ClearBlade)
cmdQueueManager.js	# 5KB – command queue manager
utils.js	# 5.5KB – Date helpers, formatters
email.js	# Email sender
utils/	# 🔧 Framework setup
admin.js	# Firebase Admin SDK init (Firestore, 2x RTDB, FCM, BigQuery)
auth.js	# 11KB – authentication + authorization
config.js	# Firebase web config
validators.js	# Input validation
triggers/	# ⚡ DATABASE TRIGGERS
coil-devices.js	# 5KB – khi coil value thay đổi → MQTT → Gateway
excel_templates/	# 📊 Excel templates (header công ty, logo)
certs/	# 🔒 SSL certificates
service_key/	# 🔑 Service account keys
image/	# 🖼️ Report images
navis-app/	# Firebase Hosting
docs/	# API documentation
generate-token.js	# JWT token generator
api-template.html	# API docs template

API Structure (Express)

```
// index-v2.js – Clean modular (thay thế file gốc 17,893 dòng)
const app = express();
app.use(cors({ origin: true }));
app.use(express.json({ limit: '10mb' }));

// Mount all 109 routes
const allRoutes = require('./routes/all-routes');
allRoutes(app);

// Export 4 functions với config khác nhau
exports.HttpPostRequestV2 = onRequest({ region: 'asia-east2', memory: '256MiB',
concurrency: 5, maxInstances: 3 }, app);
exports.HttpPostRequestV3 = onRequest({ region: 'us-central1', memory: '256MiB',
```

```

concurrency:30, maxInstances:15 }, app);
exports.httpexportexcel = onRequest({ region:'us-central1', memory:'1GiB' }, app);
// Heavy Excel export
exports.HttpPostRequest = functions.region('asia-east2').https.onRequest(app);
// v1 legacy

```

Deploy

```

# Deploy functions
firebase deploy --only functions

# Deploy hosting (navis-app)
firebase deploy --only hosting:navis-app

# Local testing
firebase emulators:start --only functions

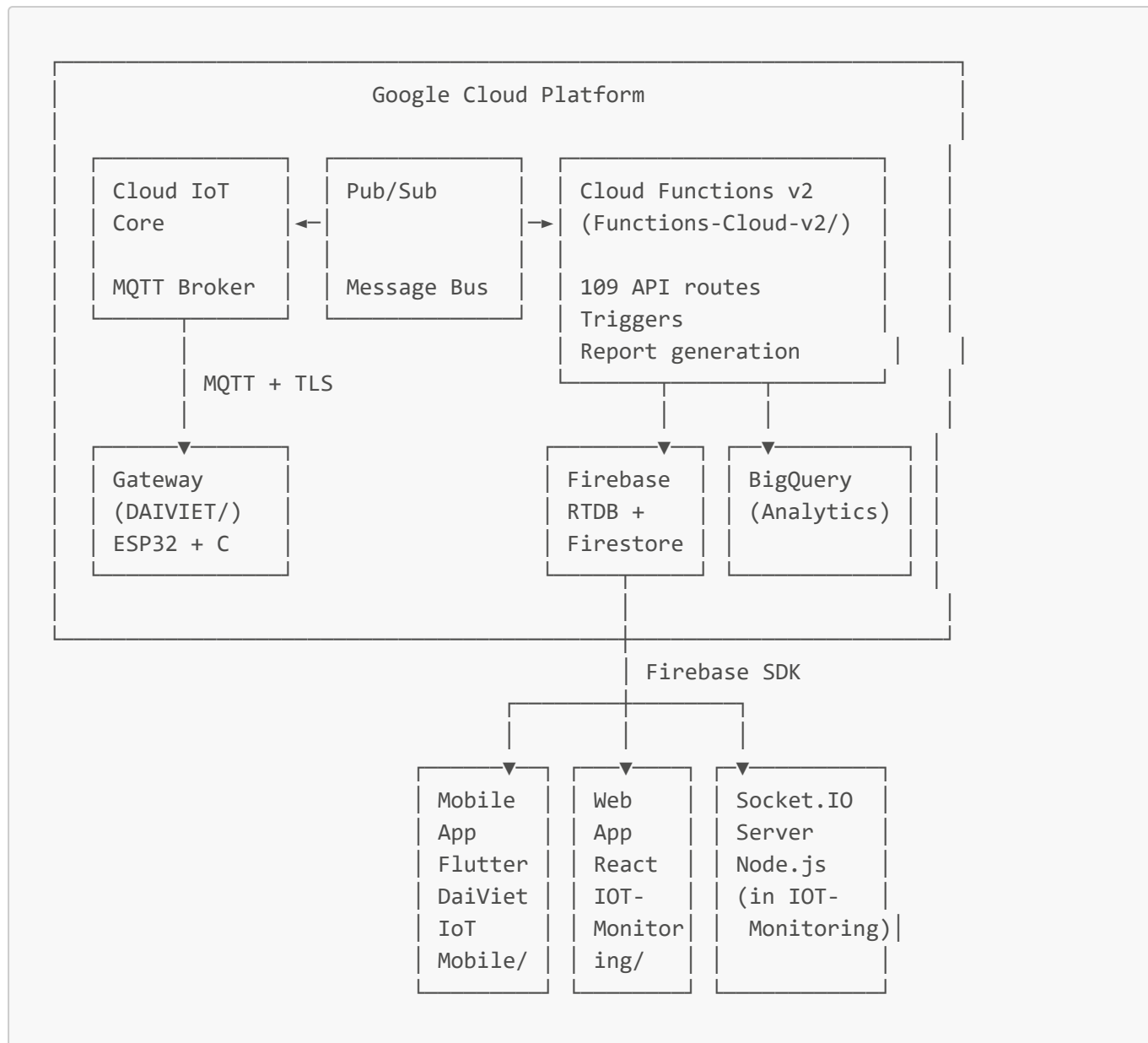
# View logs
firebase functions:log

```

Data Flow (Backend)

1. Gateway → MQTT → Google Cloud IoT Core / ClearBlade → Pub/Sub → Cloud Function
→ Parse JSON → Write to Firebase RTDB + Firestore + BigQuery
2. App/Web → REST API (109 routes) → Firebase RTDB/Firestore → JSON response
3. Alarm trigger:
Data vượt ngưỡng → helpers/alarms.js (multi-level: High1/High2/Low1/Low2)
→ FCM push notification
→ Email (SendGrid + Nodemailer)
→ Zalo ZNS (node-zalo-bot)
4. Report:
Scheduled / On-demand → helpers/excel-v2.js + report-excel.js
→ ChartJS server-side render (chartjs-node-canvas)
→ Upload to Cloud Storage → Email attachment
→ Function httpexportexcel (1GB RAM cho heavy reports)
5. Remote Control:
App → API → helpers/iot-commands.js → ClearBlade IoT → MQTT → Gateway → Modbus write

12.6 Quan hệ giữa các Repo



Tài liệu này được tạo tự động và có thể cần cập nhật khi hệ thống có thay đổi.